

Google Play App Store
DATA SCIENCE REPORT



Google Play

by Kadir Arslan

Created: 15.12.2024

Last Edited: 16.03.2026

Google Play App Store Report

The Google Play Store is a vast, dynamic ecosystem with millions of apps that cover virtually every user's need. However, for developers seeking the best strategy to launch their new apps and generate revenue from them, navigating this highly competitive market can be a challenging task.

In this report, by analyzing a comprehensive dataset comprising thousands of Android apps, we identify general trends regarding category competitiveness, pricing models, and revenue potential.

Ultimately, these data-driven insights will help clarify the store's financial situation and guide strategic decision-making regarding future application development efforts.

In this module, we will compare thousands of apps in the Google Play Store answering the following questions:

- How competitive are different app categories (Games, Lifestyle, Weather...) ?
- Which app category offers particularly promising opportunities due to its popularity?
- How many downloads would you give up by making your app paid vs. free ?
- How much can you reasonably charge for a paid app ?
- Which paid apps have generated the highest revenue ?
- How many paid apps will recoup their development costs based on their sales revenue ?
- And more...

Source Data Files (CSV Format):

Apps: github.com

Imported Libraries:

```
import pandas as pd
import plotly.express as px
import plotly.graph_objects as go
import math
```

Standardizing Numeric Output Format for Better a Readability

Before going into the analysis lets format the display of the numeric data for a more convenient readability.

```
# Show numeric output in decimal format e.g., 2.15
pd.options.display.float_format = '{:,.2f}'.format
```

This function will display all floating-point numbers in a consistent decimal format with two decimal places, making tables easier to read and interpret.

Uploading and Displaying the DataFrame:

```
df_apps = pd.read_csv('csv_files/apps.csv')
print(df_apps.shape)
print(df_apps.head())
```

Shape of DataFrame:

```
(10841, 12)
```

Head (first five rows) of the DataFrame:

	App	...	Android_Ver
0	Ak Parti Yardım Toplama	...	4.1 and up
1	Ain Arabic Kids Alif Ba ta	...	3.0 and up
2	Popsicle Launcher for Android P 9.0 launcher	...	4.2 and up
3	Command & Conquer: Rivals	...	Varies with device
4	CX Network	...	4.1 and up

```
[5 rows x 12 columns]
```

Sampling Random Data from a DataFrame:

To retrieve random rows from a DataFrame, the `.sample(n)` method is used, where `n` specifies the number of rows to return (the default value is 5).

This method returns a random subset of the data and is therefore well-suited for quickly reviewing and understanding the structure and content of the DataFrame.

```
print(df_apps.sample(6))
```

Random Data:

	App	...	Android_Ver
3625	R Programming Solution	...	4.0 and up
2101	Victoria, BC Tour City	...	4.0 and up
10092	CATS: Crash Arena Turbo Stars	...	4.1 and up
3410	infirmiers	...	4.0.3 and up
8598	Bus Driver 3D: Hill Station	...	4.1 and up

As illustrated above, the DataFrame has 10841 rows and 12 columns.

Capturing the names of columns – Features of the DataFrame:

To get a clear overview, we would like to know the names of the columns, as they represent the parameters of the DataFrame.

For this purpose, we use `.columns()` function.

```
print("\nColumns (Parameters) of the Dataframe:")
print(df_apps.columns)
```

Columns (Parameters) of the DataFrame:

```
Index(['App', 'Category', 'Rating', 'Reviews', 'Size_MBs', 'Installs', 'Type',
       'Price', 'Content_Rating', 'Genres', 'Last_Updated', 'Android_Ver'],
```

```
dtype='str')
```

So, we can see that 12 different features were originally scraped from the Google Play Store and ready to be examined as part of this Analysis.

About the Source:

Certainly, this data represents only a sample of the total number of Android apps. It does not include all Android apps, of which there are millions.

We are going to conduct our analysis by using this sample as the representative of the App Store as a whole. However, this is not necessarily the case, as this sample was compiled during the web scraping process based on the geographic location and certain retrieve methods.

The data was collected around 2017–2018.

The price data reflects the price in U.S. dollars at the time the data was collected.

The installation figures do not reflect the exact number of installations. If an app has 245,239 installations, Google simply provides an estimate such as “100,000+”.

The “+” sign has been removed from the CSV file, and for simplicity, the remaining numbers are assumed to represent the exact number of installations in this column.

Dropping (Removing) certain Columns from the DataFrame:

Since we will not be using two parameters in the analysis, we are going to remove this unwanted data to ensure a clear analysis.

The function `.drop()` will be used to remove the columns.

```
# Dropping columns:  
df_apps.drop(['Last_Updated', 'Android_Ver'], axis=1, inplace=True)
```

To remove unnecessary columns from the DataFrame, a list of column name is passed to the `.drop()` method. By specifying `axis=1`, the operation explicitly targets

columns rather than rows, ensuring that the selected variables are removed from the dataset.

Handling Missing Values in the DataFrame

Since we are going to perform an analysis based on the “Rating” variable, it is essential to exclude entries with missing values in this field.

Therefore, rows containing NaN values in the “Rating” column should be removed to ensure the accuracy and reliability of the analysis.

```
# NaN:
nan_values = df_apps[df_apps.Rating.isna()]
print(nan_values.isna().values.sum())

>> 1475
```

As can be observed the has NaN values at 1475 entries at the “Rating” section.

Displaying the related data:

```
print(nan_values)
df_apps_clean = df_apps.dropna()
```

NaN values:

	App	Category	Rating	Reviews	Size_MBs	Installs	Type	Price	Content_Rating	Genres
0	Ak Parti Yardım Toplama	SOCIAL	NaN	0	8.70	0	Paid	\$13.99	Teen	Social
1	Ain Arabic Kids Alif Ba ta	FAMILY	NaN	0	33.00	0	Paid	\$2.99	Everyone	Education
2	Popsicle Launcher for Android P 9.0 launcher	PERSONALIZATION	NaN	0	5.50	0	Paid	\$1.49	Everyone	Personalization
3	Command & Conquer: Rivals	FAMILY	NaN	0	19.00	0	NaN	0	Everyone 10+	Strategy
4	CX Network	BUSINESS	NaN	0	10.00	0	Free	0	Everyone	Business
...	...	...	...	...	...	...	...	...	...	...
5840	Em Fuga Brasil	FAMILY	NaN	1317	60.00	100,000	Free	0	Everyone	Simulation
5862	Voice Tables - no internet	PARENTING	NaN	970	71.00	100,000	Free	0	Everyone	Parenting
6141	Young Speeches	LIBRARIES_AND_DEMO	NaN	2221	2.40	500,000	Free	0	Everyone	Libraries & Demo
7035	SD card backup	TOOLS	NaN	142	3.40	1,000,000	Free	0	Everyone	Tools
7175	Android TV Remote Service	TOOLS	NaN	1	3.70	1,000,000	Free	0	Everyone	Tools

And we are going to discard this part from the data:

```
df_apps_clean = df_apps.dropna()
```

Dealing with Duplicates in the DataFrame:

To ensure a reliable and accurate analysis of the situation, it is essential to remove duplicate entries from the dataset before examining the conditions.

Here, `.duplicated()` function is used to identify duplicate .

The results are displayed according to a conditional display scheme(double square brackets) to ensure that only the relevant columns are selected and displayed.

```
# Displaying the duplicated rows
duplicated_rows = df_apps_clean[df_apps_clean.duplicated()]
print("\nDuplicated rows:")
print(duplicated_rows[['App', 'Size_MBs', 'Type', 'Price']].head())
```

Duplicated rows:

	App	Size_MBs	Type	Price
946	420 BZ Budeze Delivery	11.00	Free	0
1133	MouseMingle	3.90	Free	0
1196	Cardiac diagnosis (heart rate, arrhythmia)	6.50	Paid	\$12.99
1231	Sway Medical	22.00	Free	0
1247	Chat Kids - Chat Room For Kids	4.90	Free	0

Selective removing - Determining the identity of the Data:

With large DataFrames, such as the one used in this analysis, it is difficult to visually distinguish individual rows from one another. Identical or duplicate data entries may appear in multiple records with minor differences in other attributes.

Therefore, identifying and removing duplicate records is a critical step in preprocessing to ensure data integrity.

As an example, records for 'Instagram':

```
# Filtering the DataFrame to obtain the desired Section:
df_instagram = df_apps_clean[df_apps_clean.App == 'Instagram']
print("\nInstagram data:")
print(df_instagram)
```

Instagram data:

	App	Category	Rating	Reviews	Size_MBs	Installs	Type	Price	Content_Rating	Genres
10806	Instagram	SOCIAL	4.50	66577313	5.30	1,000,000,000	Free	0	Teen	Social
10808	Instagram	SOCIAL	4.50	66577446	5.30	1,000,000,000	Free	0	Teen	Social
10809	Instagram	SOCIAL	4.50	66577313	5.30	1,000,000,000	Free	0	Teen	Social
10810	Instagram	SOCIAL	4.50	66509917	5.30	1,000,000,000	Free	0	Teen	Social

Here, Instagram's data rows are not recognized as duplicates by the standard *deduplicated()* function because they contain different values in different fields.

Therefore, we will develop a method to remove the duplicate data.

Determining the components for duplicate values and removing the values based on these criteria:

We are going to Remove the duplicated data using subset which identifies the “actual” data.

Here, we are going to choose the name, type and prices of an application as an identifier of that application. If three of these values are equal in multiple rows, these rows will be considered as identical.

```
identity_of_apps = ['App', 'Type', 'Price']  
df_apps_clean = df_apps_clean.drop_duplicates(subset=identity_of_apps)
```

Displaying the clean data:

```
df_instagram_clean = df_apps_clean[df_apps_clean.App == 'Instagram'].head()  
print("\nClean Instagram Data:")  
print(df_instagram_clean)
```

Clean Instagram Data:

	App	Category	Rating	Reviews	Size_MBs	Installs	Type	Price	Content_Rating	Genres
10806	Instagram	SOCIAL	4.50	66577313	5.30	1,000,000,000	Free	0	Teen	Social

Clean DataFrame

```
print(df_apps_clean.shape)
```

```
>> (8199, 10)
```


Now we have eliminated all the NaN and duplicated data from the DataFrame.

Sorting Data within a DataFrame to Identify Records with Specific Attributes

```
# Sorting
sorted_by_rating = df_apps_clean.sort_values('Rating', ascending=False)
sorted_by_size = df_apps_clean.sort_values('Size_MBs', ascending=False)
sorted_by_review = df_apps_clean.sort_values('Reviews', ascending=False)
```

Receiving most popular apps in the DataFrame:

Here we are going to search for the most 10 popular apps.

Since the data has 12 columns, here we are going to pass on desirable parameters into to the function for displaying.

```
print(sorted_by_review.head(10)[['App', 'Category', 'Rating', 'Reviews', '']])
```

	App	Category	Rating	Reviews	Type
10805	Facebook	SOCIAL	4.10	78158306	Free
10785	WhatsApp Messenger	COMMUNICATION	4.40	69119316	Free
10806	Instagram	SOCIAL	4.50	66577313	Free
10784	Messenger - Text and Video Chat for Free	COMMUNICATION	4.00	56642847	Free
10650	Clash of Clans	GAME	4.60	44891723	Free
10744	Clean Master- Space Cleaner & Antivirus	TOOLS	4.70	42916526	Free
10835	Subway Surfers	GAME	4.50	27722264	Free
10828	YouTube	VIDEO_PLAYERS	4.30	25655305	Free
10746	Security Master - Antivirus, VPN, AppLock, Boo...	TOOLS	4.70	24900999	Free
10584	Clash Royale	GAME	4.60	23133508	Free

An examination of the usage statistics shows which apps are used most frequently on the Android platform, with established services such as Facebook, WhatsApp Messenger, and Instagram taking the top spots.

A high number of reviews indicates strong user engagement and broad market penetration.

Notably, all the highest-rated apps in this subgroup are offered as free services, suggesting a strong correlation between accessibility and user reach. This pattern indicates the dominance of free-to-use models among the most popular apps in the dataset.

Content Distribution of Applications

In Google Store, each application is assigned a content rating such as Everyone, Teen, or Mature 17+ to indicate its suitability for different audiences. Understanding the distribution of these content ratings within a dataset provides valuable insights into the overall target audience and the content characteristics of the applications being analyzed.

In this section, we are going to explore how these ratings are distributed across the dataset and discuss appropriate methods for visualizing this distribution.

Here, we are going to deploy Plotly, a widely used data visualization library known for its interactivity and design flexibility. We are going to construct a pie chart using Plotly which allows for a clear and intuitive representation of proportional data. In Plotly, all visual representations like pie charts, bar charts, or line charts are treated as chart objects, ensuring consistency and extensibility across different visualization types.

Formulating a Convenient Dataset:

We are utilizing here `value_counts()` method from Pandas.

This method is used to count the frequency of unique values in a series (or a single DataFrame column). It returns a sorted series where the index contains the unique values and the corresponding values indicate their frequency, sorted by default in descending order of frequency.

This makes it particularly well-suited for quickly understanding the distribution of categorical data.

```
ratings = df_apps_clean.Content_Rating.value_counts()
```

Generating Pie Chart

Code for the graph:

```
fig = px.pie(labels=ratings.index,  
             values=ratings.values,  
             title="Content Rating",
```

```

        names=ratings.index,
        hole=0.4,
    )

fig.update_traces(textposition='outside', textinfo='percent+label',
textfont_size=13)

fig.update_layout(
    title=dict(
        text="Content Rating Distribution",
        font=dict(size=22, family="Arial, sans-serif", color='#2C3E50'),
        x=0.5, y=0.95
    ),

    # Adding the text inside the donut hole
    annotations=[dict(
        text='Ratings',
        x=0.5, y=0.5,
        font=dict(size=18, color='#7F8C8D', family="Arial"), showarrow=False
    )],

    legend=dict(
        font=dict(size=13, color='#34495E'),
        orientation="v",
        x=1.05, y=0.5
    ),

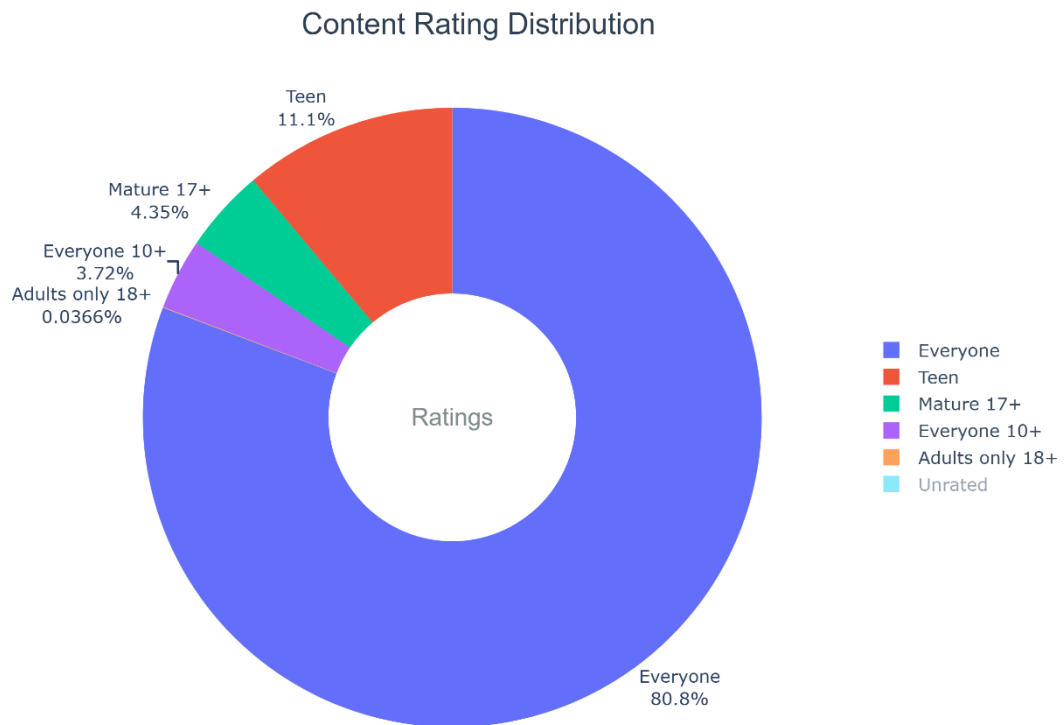
    showlegend=True,
    margin=dict(t=100, b=40, l=40, r=40),
    paper_bgcolor='white',
    plot_bgcolor='white'
)

# Download options
download_config = {
    'toImageButtonOptions': {
        'format': 'png',
        'filename': 'Content_Rating_Pie',
        'width': 900, 'height': 600,
        'scale': 3
    }
}

fig.show(config=download_config)

```

Pie Chart:



Interactive Graph Link: [Content Rating Distribution](#)

The figure shows a clear predominance of apps aimed at a broad audience, with the “Everyone” category accounting for approximately 80.8% (6,621 apps) of the dataset. This suggests that most Android apps are designed to be accessible to all users.

The “Teen” category accounts for a significantly smaller share at 11.1% (912 apps), suggesting that there is a moderate number of apps in this category that, while dealing with slightly more mature topics, still appeal to a broad user base.

More restrictive categories such as “Mature 17+” (4.35%) and “Adults Only” (0.0366%) account for only a small portion of the dataset, which illustrates that content with high age ratings or explicit content is relatively rare on the platform.

Overall, the distribution shows a clear trend in the Android app market toward inclusive, family-friendly content and away from niche applications or age-restricted apps.

Preparing Numeric Data in the DataFrame:

As we have seen before we have commas in 'Installs' column which shows the number of installments of an application. We would like to handle this commas to enable a numeric data in that section.

Inspection of the column with the function `.describe()`

This function generates a brief statistical summary of the numeric columns in a dataset, including the count, mean, standard deviation, minimum, maximum, and quartiles.

It helps us to understand the distribution and dispersion of the data at a glance; when applied to non-numeric columns, categorical summaries (such as top entries and frequency counts) can also be displayed.

```
print(df_apps_clean.Installs.describe())
```

```
count      8199
unique       19
top      1,000,000
freq       1417
Name: Installs, dtype: object
```

The presence of commas prevents the numerical analysis of the data. To resolve this, the commas can be removed from the string using the `.replace()` method, replacing them with an empty string. After cleaning the data, the column can be converted to a numeric type using `pd.to_numeric()`, which enables correct analysis.

`.loc[]` is a label-based indexing method in Pandas that allows rows and columns to be selected based on their labels, enabling precise data access and correction.

`pd.to_numeric()` is a Panda function that converts data to a numeric type (such as integers or floating-point numbers) and handles non-numeric values according to specified parameters (e.g., by converting errors to NaN).

```
df_apps_clean.loc[:, 'Installs'] =  
    df_apps_clean['Installs'].astype(str).str.replace(',', '')  
  
df_apps_clean.Installs = pd.to_numeric(df_apps_clean.Installs)
```

Same process for the “Price” Parameter:

```
print(df_apps_clean.Price.describe())
```

```
count      8199
unique       73
top         0
freq       7595
Name: Price, dtype: object
```

```
df_apps_clean.loc[:, 'Price'] =
    df_apps_clean['Price'].astype(str).str.replace('$', '')

df_apps_clean.Price = pd.to_numeric(df_apps_clean.Price)
```

Ensuring the Integrity and Accuracy of the Data

Several apps in the App Store do not provide reliable or trustworthy information. To improve the quality and integrity of the dataset, these apps should be removed from the DataFrame. This filtering step ensures that the subsequent analysis is based on more credible data.

Next, we will examine the most expensive apps in the dataset

```
df_apps_clean = df_apps_clean.sort_values('Price', ascending=False)
print(df_apps_clean.head(6))
```

	App	Category	Rating	Reviews	Size_MBs	Installs	Type	Price	Content_Rating
3946	I'm Rich - Trump Edition	LIFESTYLE	3.60	275	7.30	10000	Paid	400.00	Everyone
2394	I am Rich!	FINANCE	3.80	93	22.00	1000	Paid	399.99	Everyone
2461	I AM RICH PRO PLUS	FINANCE	4.00	36	41.00	1000	Paid	399.99	Everyone
1331	most expensive app (H)	FAMILY	4.30	6	1.50	100	Paid	399.99	Everyone
3145	I am rich(premium)	FINANCE	3.50	472	0.94	5000	Paid	399.99	Everyone
2775	I Am Rich Pro	FAMILY	4.40	201	2.70	5000	Paid	399.99	Everyone

The results suggest that these entries do not represent legitimate applications suitable for analysis.

A notable example is the group of “I Am Rich” apps, which are offered at exceptionally high prices (often \$300 or more) without providing any functional benefit. The original concept dates back to 2008, when Armin Heinrich released a similar app in the iOS App Store for \$999.90, which served merely as a stunt to signal wealth. Although it generated few sales before its removal, it inspired numerous imitators in the Google Play Store.

These apps are considered non-representative for analytical purposes, and their unusually high installation numbers may be artificially inflated, possibly due to a temporary free availability aimed at generating reviews and credibility.

Selective Data Filtering for Improved Accuracy and Reliability

We are going to remove the apps which cost more than 250 \$ to obtain a clear Base.

Here, we can simply use pandas convenient square-brackets filtering technique.

```
df_apps_clean = df_apps_clean[df_apps_clean['Price'] < 250]
df_apps_clean = df_apps_clean.sort_values('Price', ascending=False)
```

Creating a new Column (Parameter) based on existing data.

To better capture the financial performance of the applications, we would like to create a new column for “Estimated Revenues” of the applications.

In the DataFrame we have the number of installments and the price of the apps, so we can easily generate this column based on these two metrics.

In pandas, this is achieved by assigning a new column using bracket notation, while the `.mul()` function is applied to perform element-wise multiplication between the two numeric variables (previously converted to appropriate data types).

```
df_apps_clean['Revenue_Estimate'] =
    df_apps_clean.Installs.mul(df_apps_clean.Price)
```

Displaying:

Let's take a look at the 10 apps with the highest revenue:

```
df_apps_clean = df_apps_clean.sort_values('Revenue_Estimate', ascending=False)

print(df_apps_clean[['App', 'Price', 'Category', 'Revenue_Estimate']].head(10))
```

	App	Price	Category	Revenue_Estimate
9220	Minecraft	6.99	FAMILY	69,900,000.00
8825	Hitman Sniper	0.99	GAME	9,900,000.00
7151	Grand Theft Auto: San Andreas	6.99	GAME	6,990,000.00
7477	Facetune - For Free	5.99	PHOTOGRAPHY	5,990,000.00
7977	Sleep as Android Unlock	5.99	LIFESTYLE	5,990,000.00
6594	DraStic DS Emulator	4.99	GAME	4,990,000.00
6082	Weather Live	5.99	WEATHER	2,995,000.00
6746	Card Wars - Adventure Time	2.99	FAMILY	2,990,000.00
7044	Tasker	2.99	TOOLS	2,990,000.00
7633	Five Nights at Freddy's	2.99	GAME	2,990,000.00

The results clearly show that Minecraft, with estimated revenue of nearly \$70 million, is far ahead of the pack, leaving all other apps in the dataset far behind. This is a testament not only to its popularity, but also to the effectiveness of its pricing strategy and its large user base.

Interestingly, although Minecraft is essentially a game, it is classified under “Family”; however, when viewed as a game, the top-3-grossing apps fall into the “Games” category. Overall, 7 of the 10 highest-grossing paid apps belong to the gaming sector (including those listed in non-gaming categories), which supports the assumption that users are more willing to pay for interactive entertainment experiences.

Other categories such as Photography, Lifestyle, and Tools appear in the list indicates also a huge monetization potential for paid apps outside the gaming sector.

Analysis for Categories

Number of Applications by Category

To analyze performance at the category-level performance, we use a combination of bar charts and scatter plots to identify patterns and trends across the dataset.

This analysis allows us to assess which application categories have the strongest market presence and the greatest revenue potential. By visually comparing key metrics, we can identify the categories that dominate the market and gain insight into their relative performance.

This gives number of unique values:

```
print(df_apps_clean.Category.nunique())  
  
>> 33
```

As we see, there are 33 different categories to qualify different applications on Android App Store.

Creating Bar Graph:

We are going to create a bar graph which shows the 10 most popular categories with the greatest number of applications in these categories in play store.

Forming the DataFrame:

```
top10_category = df_apps_clean.Category.value_counts()[ :10]
```

Here, *value_counts()* computes the frequency of each unique category in the dataset, while *[:10]* slices the result to return only the top 10 most frequent categories.

Code for the graph:

```
fig = px.bar(  
    x=top10_category.index,  
    y=top10_category.values,
```

```

orientation='v', # Vertical alignment
title="Top 10 App Categories",
text=top10_category.values, # Add data labels directly to bars
color=top10_category.index, # To make very distinguished colors
color_continuous_scale='Agsunset',
)

# Updating the layout for a elegant look
fig.update_layout(
    plot_bgcolor='rgba(0,0,0,0)', # Transparent background
    paper_bgcolor='rgba(0,0,0,0)',
    font=dict(family="Helvetica, Arial, sans-serif", size=12, color="#333333"),
    title_font=dict(size=24, color="#111111", family="Helvetica, Arial, sans-
serif"),
    margin=dict(l=0, r=20, t=60, b=20),
    xaxis_title="Categories",
    yaxis_title="Number of Apps",
    xaxis_title_font=dict(size=14, family='Arial Black'),
    yaxis_title_font=dict(size=14, family='Arial Black'),
    coloraxis_showscale=True, # Hide color scale
    showlegend=False # Dont show the Legend
)

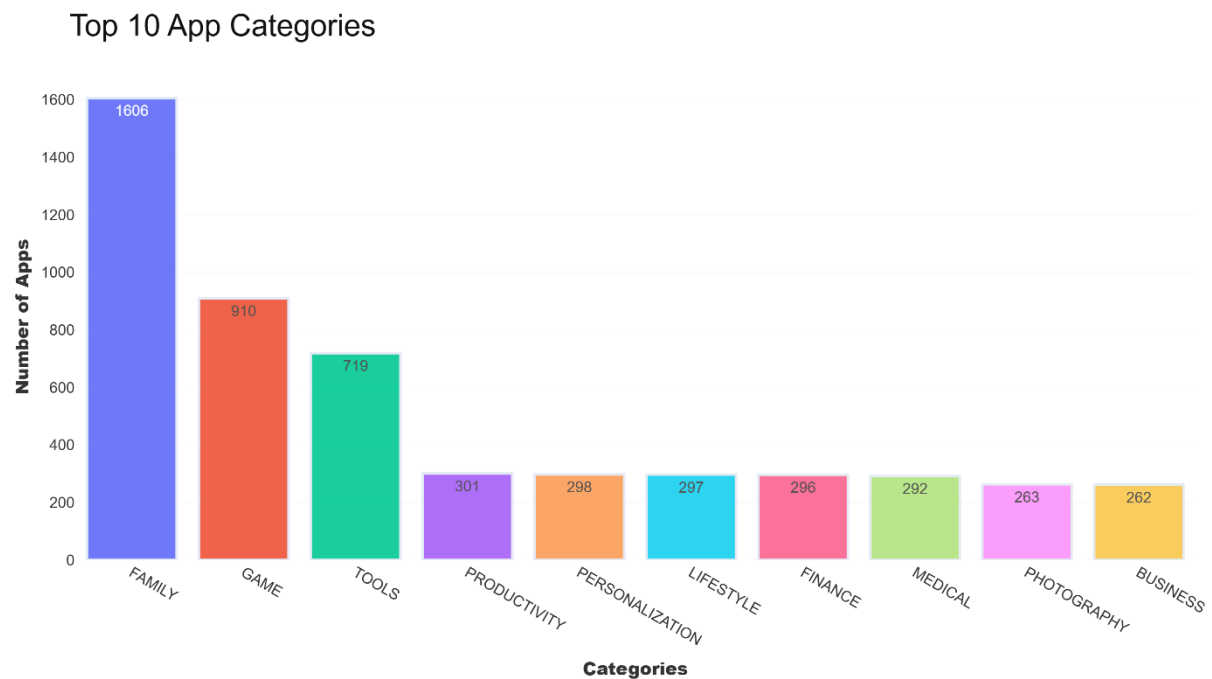
# Adding a little bit of opacity.
fig.update_traces(opacity=0.9)

download_config = {
    'toImageButtonOptions': {
        'format': 'png',
        'filename': 'top_10_categories',
        'width': 960, 'height': 540,
        'scale': 3
    }
}

fig.show(config=download_config)

```

Bar Graph:



Graph Link: [Top 10 App Categories](#)

The outcomes show that the “Family” category ranks first by a wide margin, followed by ‘Games’ and “Tools,” indicating a strong concentration of apps in these segments.

This suggests that these categories are highly saturated compared to others, such as “Business” or “Photography,” which have significantly fewer apps.

The findings also highlight that applications in Google Play store are shaped mainly by daily uses and entertainment purposes rather than information sources or self-development services.

In terms of the number of apps, the “Family” and “Games” categories are the most competitive, making it difficult for new providers to stand out from the crowd. In contrast, less crowded categories may offer better opportunities for visibility and market penetration.

Number of Installations by Category

While analyzing the total number of apps per category provides insight into market competition, it does not fully capture user demand. It is more informative to examine how frequently apps within each category are downloaded. This allows us to better understand relative popularity and user interest across the various categories.

To achieve this, we are going to form a dataset grouped by category, and the total number of installations is aggregated for each group.

Applying Aggregation Functions on Grouped Data

```
cat_number = df_apps_clean.groupby('Category').agg({'App': pd.Series.count})

category_installs = df_apps_clean.groupby('Category').agg({'Installs':
pd.Series.sum})
```

In this step, the dataset is grouped by the Category variable using *groupby()*, which divides the data into subsets, with each group representing a specific app category.

Next, *agg()* function is applied to calculate summary statistics for each group: *pd.Series.count* counts the number of apps in each category, while *pd.Series.sum* calculates the total number of installations per category.

This approach allows for a structured comparison of both the number of apps and their overall download performance across all categories.

Creating Horizontal Graph:

We are going to create a horizontal bar graph showing the most 20

```
category_installs_sorted = category_installs.sort_values(by='Installs',
ascending=True)[-20:]
```

This line uses *sort_values()* to sort the categories in ascending order by their total number of installations and then selects the 20 categories with the most installations by using *[-20:]* to select the last 20 rows.

Code for the horizontal graph:

```
h_bar = px.bar(
    x=category_installs_sorted.Installs,
    y=category_installs_sorted.index,
    orientation='h',
    title='<b>App Category Popularity by Downloads</b>',
    text_auto='.2s', # Automatically formats large numbers (e.g., 1.5B, 500M)
    color_continuous_scale="Teal",
    color=category_installs_sorted.Installs
)

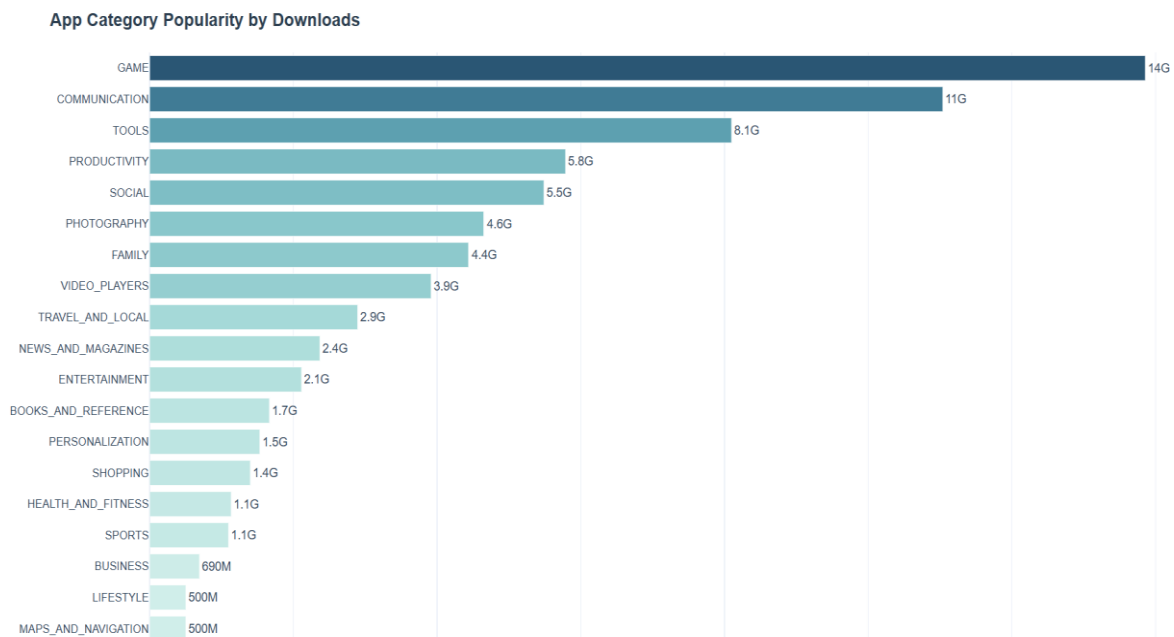
h_bar.update_layout(
    template='plotly_white', # A clean white background
    xaxis_title='',
    yaxis_title='',
    font=dict(family='Inter, Helvetica, Arial, sans-serif', size=13,
    color='#34495E'),
    title_font=dict(size=20, color='#2C3E50'),
    margin=dict(l=0, r=40, t=60, b=0),
    height=800,
    showlegend=False,
    coloraxis_showscale=False
)

h_bar.update_traces(
    textfont_size=14, textangle=0, textposition="outside",
    cliponaxis=False # Ensures labels at the very edge don't get cut off
)

download_config = {
    'toImageButtonOptions': {
        'format': 'png',
        'filename': 'app_category_by_downloads',
        'width': 1920, 'height': 1920, 'scale': 3
    }
}

h_bar.show(config=download_config)
h_bar.write_html("interactive_graphs/apps/app_category_by_downloads.html")
```

Graph:



Graph link: [App Category Popularity by Downloads](#)

The chart illustrates that the Game and Communication categories lead by a significant margin in total installations with 14 and 11 billion downloads, underscoring their widespread popularity and strong user engagement.

Categories such as “Tools,” “Productivity,” “Social Networks,” and “Photography” also boast high numbers, with over a billion downloads each, suggesting that they fulfill important and frequently used functions in users’ daily lives.

Meanwhile, categories such as “Business,” “Lifestyle,” “Finance,” and “Maps & Navigation,” have relatively lower overall figures, suggesting more specialized use cases or a smaller target audience.

Overall, the distribution suggests that entertainment and utility-driven apps dominate user demand on the Google Play Store.

Comparative Analysis of Categories through App and Installation Numbers

Although previous findings have shown that categories such as “Games” and “Tools” dominate total installations, this alone does not reveal how downloads are distributed within each category.

To gain a deeper understanding, it makes sense to analyze the relationship between the number of apps (supply) and the total number of installations (demand) at the same time.

By visualizing these variables in a scatter plot, we can assess how concentrated each category is—whether a small number of apps account for the majority of downloads or whether user activity is more evenly distributed across many apps.

Merging DataFrames :

First, we are going to merge two different datasets which we used in previous two analysis into a bigger DataFrame.

```
cat_merged_df = pd.merge(cat_number, category_installs, on='Category',  
how="inner")  
  
cat_merged_df = cat_merged_df.sort_values('Installs', ascending=False)
```

Here, the two aggregated DataFrames—one containing the number of apps per category and the other containing the total number of installations—are combined into a single DataFrame using `merge()`.

The merge() function joins the two datasets based on the common “Category” column (where “how=‘inner’” ensures that only matching categories are included), so that both metrics can be analyzed together.

Finally, the combined DataFrame is sorted in descending order by total number of installations to highlight the most popular categories.

Displaying the new merged Dataset:

```
print(cat_merged_df.head())
```

Category	App	Installs
GAME	910	13858762717
COMMUNICATION	257	11039241530
TOOLS	719	8099724500
PRODUCTIVITY	301	5788070180
SOCIAL	203	5487841475

The dataset presents the categories with total numbers of App's and Installs.

Creating Scatter Plot

We are going to create an interactive scatter plot which illustrates the both parameters in single chart.

code for the scatter graph:

```
scatter = px.scatter(
    cat_merged_df,
    x='App',
    y='Installs',
    title='Category Concentration: Apps vs. Installs',
    size='App',
    size_max=45,
    hover_name=cat_merged_df.index,
    color=cat_merged_df.Installs,
    color_continuous_scale=px.colors.sequential.Teal, # Modern, clean color
    gradient
    template='plotly_white' # Removes the default gray background
)

# Updating traces to add professional borders and opacity to the bubbles
scatter.update_traces(
    marker=dict(
        line=dict(width=1, color='DarkSlateGrey'), # Crisp borders around bubbles
        opacity=0.75
    )
)

scatter.update_layout(
    # Center the title and use a clean, modern font
    title=dict(
```



```

        font=dict(size=25, family="Helvetica Neue, Helvetica, Arial, sans-serif",
color='#2b5876'),
        x=0.5, xanchor='center'
    ),
    # Global font settings
    font=dict(family="Helvetica Neue, Helvetica, Arial, sans-serif", size=12,
color='#555555'),

    # X-Axis styling
    xaxis_title="Number of Apps (Lower = More Concentrated)",
    xaxis=dict(
        showgrid=True,
        gridcolor='#EBEBEB',
        gridwidth=1,
        zeroline=False # Removes the harsh zero line
    ),

    # Y-Axis styling
    yaxis_title="Total Installs",
    yaxis=dict(
        type='log',
        showgrid=True,
        gridcolor='#EBEBEB',
        gridwidth=1,
        griddash='dot',
        zeroline=False
    ),

    xaxis_title_font=dict(size=14, family='Arial Black', color='#2b5876'),
    yaxis_title_font=dict(size=14, family='Arial Black', color='#2b5876'),

    # Cleaning up the colorbar-Legend
    coloraxis_colorbar=dict(
        title="Installs",
        thicknessmode="pixels", thickness=15,
        lenmode="pixels", len=300,
        yanchor="middle", y=0.5,
        ticks="outside",
        tickformat=".2s" # Formats large numbers (e.g., 100M instead of
100,000,000)
    ),

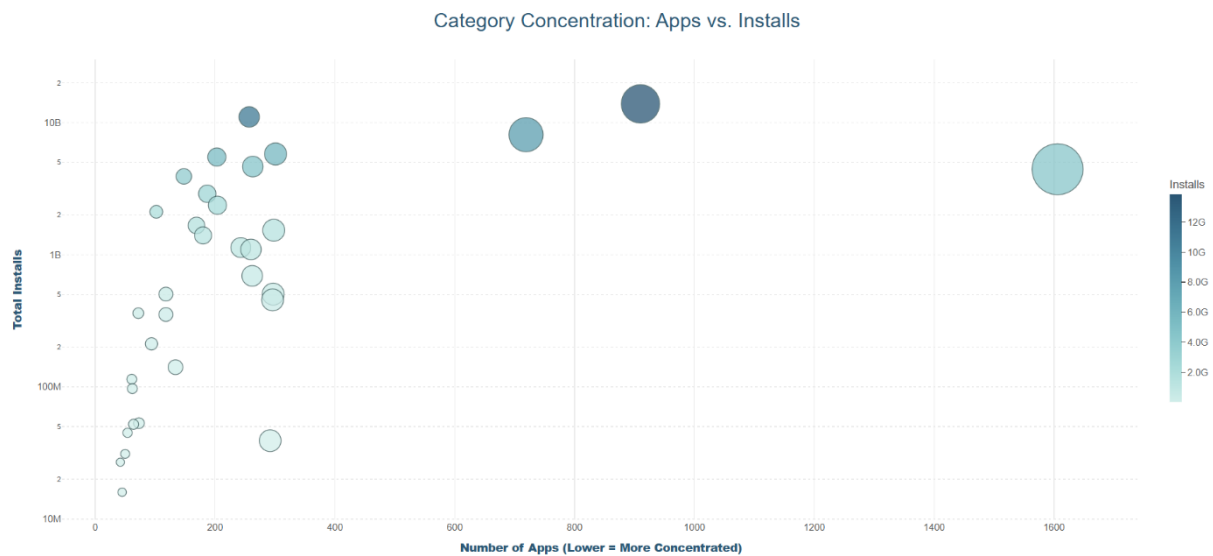
    # Add breathing room around the edges
    margin=dict(l=60, r=40, t=80, b=60)
)

scatter.show()

scatter.write_html("interactive_graphs/apps/scatter_category_downloads.html")

```

Graph



Interactive Graph Link: [Category Concentration: Apps vs. Installs](#)

The scatter plot shows a clear correlation between market size (number of apps) and user demand (total number of installations), while also illustrating how effectively each category converts its offerings into downloads.

Categories such as “Communication” and “Productivity” are characterized by the fact that, despite having a relatively small number of apps, they achieve extremely high installation figures, indicating strong dominance and high demand concentrated on a smaller number of successful applications. In contrast, categories such as “Family” and “Games” may have a very large number of apps and high total installation figures, but their size suggests intense competition, making it more difficult to gain visibility and achieve success.

Notably, categories such as “Medical” and “Lifestyle” have a relatively high number of apps but record comparatively few installations, suggesting weaker demand or limited user engagement despite a wide range of offerings. This suggests that entering oversaturated categories with low demand may not be strategically advantageous.

For new investors and developers, categories with many installations but few apps offer attractive opportunities, while highly competitive or underperforming categories should be approached with caution.

Analysis for the Genres of Apps

Understanding the distribution of app genres provides valuable insights into market trends and user preferences within the dataset. In this section we are going to examine how many different genres there are and investigate whether individual apps can belong to multiple categories.

Since genre data may be stored as nested or combined values, standard counting methods like `.value_counts()` can produce misleading results.

To address this, we apply techniques such as string splitting and DataFrame stacking to accurately analyze and represent the genre landscape.

Receiving The Number of Unique Values in a Particular Column

```
print(df_apps_clean.Genres.nunique())  
  
>> 114
```

As reflected in the data, when we search for the number of unique Genres values, we initially observe 114 unique values.

However, this figure can be misleading due to the presence of nested or combined genre entries within individual records.

We are going to utilize `.value_counts()` function to see the grouped data by genres and their numbers of appearing in the DataFrame.

The function `.tail(n)` returns the last `n` entries of the DataFrame.

```
df_genres_count = (df_apps_clean.Genres.value_counts())  
print(df_genres_count.sort_values(ascending=False).tail())
```

Genres

```
Casual;Music & Video      1  
Tools;Education          1  
Role Playing;Brain Games 1  
Adventure;Education      1  
Lifestyle;Pretend Play    1  
Name: count, dtype: int64
```

As the results show, it is clear that some entries span multiple genres and have been grouped together rather than representing a single, clearly defined category.

Focusing on entries with only one genre highlights this inconsistency and emphasizes the need for preprocessing and redesigning the dataset before an accurate genre analysis can be conducted.

Resolving Disorganized Subsets in the DataFrame

As observed in the dataset, the genre names are separated by semi-colons (;).

We need to distinguish separate the genre names to get a clear overview. For this purpose, we are going to use `split()` and `stack()` functions.

```
genres_list = df_apps_clean.Genres.str.split(';', expand=True).stack()
num_genres = genres_list.value_counts()
```

In this step, we restructure the Genres column to enable a more detailed analysis of the frequency of individual genres.

This `.split(';', expand = True)` function splits multiple genres stored in a single value into separate columns based on the delimiter.

Since this results in a wide format with multiple columns, the `.stack()` method is then used to reorganize the data into a single column by grouping these values into a vertical structure.

This conversion allows us to effectively use the `.value_counts()` function and determine the exact number of each genre across all apps.

Inspection of the new DataFrame:

```
print(len(num_genres))
print(num_genres.head())
```

53

Tools	719
Education	587
Entertainment	498
Action	304
Productivity	301

Name: count, dtype: int64

As revealed by the data, we have now a DataSet containing 53 different genres, along with the number of times each has been used in apps on the Google Play Store.

Creating the Bar Graph:

We are going to plot most popular 20 Genres on applications.

code:

```
# Lets create a graph for most 20 popular genres on App store
top_20 = num_genres[:20].sort_values(ascending=True)

bar = px.bar(
    x=top_20.values, y=top_20.index,
    orientation='h', # horizontal alignment
    title='Top 20 App Genres',
    text=top_20.values, # Direct Labeling on the bars
    color_continuous_scale=px.colors.sequential.Bluy1,
    color= top_20.values
)

bar.update_layout(
    template='plotly_white', # a white background
    title_font=dict(size=24, family='Arial, sans-serif', color='#2C3E50'),
    title_x=0.02, xaxis_title='Number of Apps', yaxis_title='',
    font=dict(family='Arial, sans-serif', size=13, color='#555555'),

    # Cleaning up the axes
    xaxis=dict(showgrid=False, showticklabels=False, zeroline=False),
    yaxis=dict(showgrid=False, ticklabelposition="outside"),
```

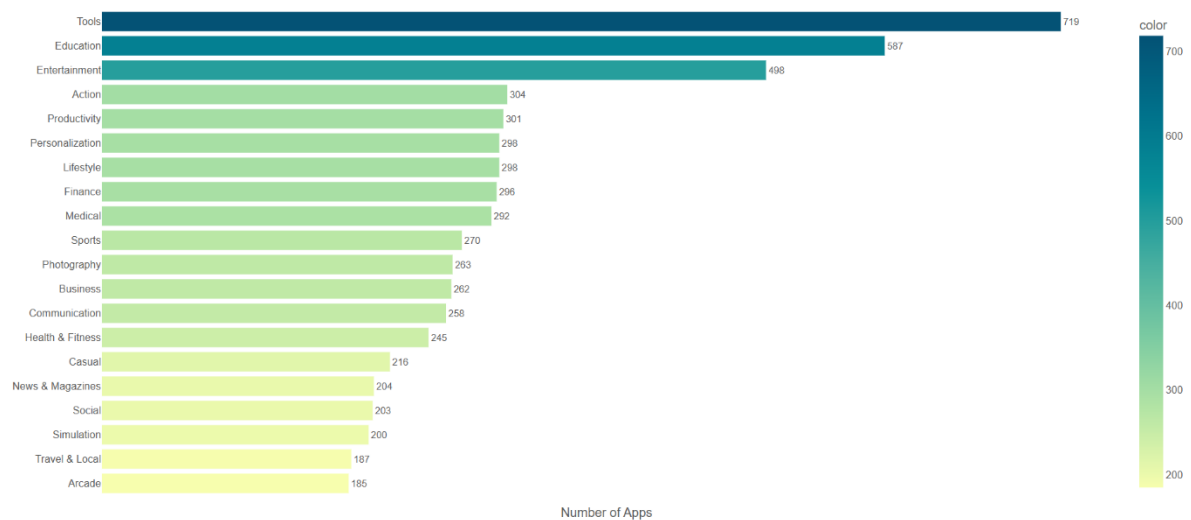
```

margin=dict(l=10, r=40, t=70, b=20)
)
# Formatting the bars
bar.update_traces(
    textposition='outside', textfont_size=12
    marker_line_width=0, # Removes the border around bars
)
bar.show()

```

Graph:

Top 20 App Genres



Graph Link: [Top 20 App Genres](#)

The distribution of app genres shows a clear dominance of practical and utility-oriented categories, with “Tools” and “Education” leading by a wide margin. This suggests that users primarily use mobile apps to boost productivity, learn, and solve everyday problems, rather than for purely entertainment-related purposes. While entertainment-related genres such as Entertainment, Action, and Casual remain popular, they are more widely distributed across various subcategories.

Overall, the data reveals a balanced ecosystem in which functional apps slightly outnumber leisure-oriented apps, reflecting both user demand and developers’ priorities.

Comparison Between Free and Paid Apps

In addition to examining the distribution of apps across different categories and genres, it is also important to analyze their pricing models.

Understanding the relationship between free and paid apps provides valuable insights into market dynamics, user accessibility, customer behaviors and monetization strategies within the app ecosystem.

Therefore, in this section we are going to focus on comparing the prevalence of free and paid apps, examining how they are distributed across different segments, and drawing conclusions about general industry trends.

```
print("\nDistribution of Apps:")
print(df_apps_clean.Type.value_counts())
```

Type	
Free	7595
Paid	589

The initial analysis shows that most apps in the Google Play Store are available for free. However, this general trend may vary by category, with certain segments potentially having a higher proportion of paid apps.

To examine this in more detail, first, we are going to group the DataSet by category and then by type, allowing for a structured comparison between free and paid apps within each category.

Next, we are going to aggregate the total number of apps for each type to the DataSet. By setting *as_index = False* in *groupby()* function, the grouped data remains in column format rather than being converted into an index, which improves readability and facilitates further analysis.

```
df_free_vs_paid =
    df_apps_clean.groupby(["Category", "Type"], as_index=False)
    .agg({'App': pd.Series.count})
```

```
print(df_free_vs_paid.shape)
print(df_free_vs_paid.head(11))
```

(61, 3)

	Category	Type	App
0	ART_AND_DESIGN	Free	58
1	ART_AND_DESIGN	Paid	3
2	AUTO_AND_VEHICLES	Free	72
3	AUTO_AND_VEHICLES	Paid	1
4	BEAUTY	Free	42
5	BOOKS_AND_REFERENCE	Free	161
6	BOOKS_AND_REFERENCE	Paid	8
7	BUSINESS	Free	253
8	BUSINESS	Paid	9

The data demonstrates that, the categories in the Dataset has now group by paid and Free showing the number of apps in each row.

Creating bar graph with two parameters

Code:

```
custom_colors = {'Free': '#2A4B7C', 'Paid': '#E07A5F'}

g_bar = px.bar(
    df_free_vs_paid,
    x='Category',
    y='App',
    color='Type',
    barmode='group',
    color_discrete_map=custom_colors, # Apply the custom palette
)

g_bar.update_layout(
    template='plotly_white',

    font=dict(family="Inter, Roboto, Arial, sans-serif", size=13,
    color="#444444"),

    title={
        'text': '<b>Free vs Paid Apps by Category</b>',
        'y': 0.95,
        'x': 0.05,
```



```

        'xanchor': 'left',
        'yanchor': 'top',
        'font': dict(size=22, color='#222222')
    },

    legend=dict(
        title=None,
        orientation='h',
        yanchor='bottom',
        y=1.02,
        xanchor='right',
        x=1
    ),

    # Streamlined X-Axis
    xaxis=dict(
        title=None, # "Categories are obvious
        categoryorder='total descending',
        tickangle=-45,
        showgrid=False,
        showline=True,
        linecolor='#DDDDDD',
        linewidth=1
    ),

    # Refined Y-Axis
    yaxis=dict(
        title='Number of Apps (Log Scale)',
        type='log',
        showgrid=True,
        gridcolor='#F0F0F0',
        zeroline=False
    ),

    hovermode='x unified',

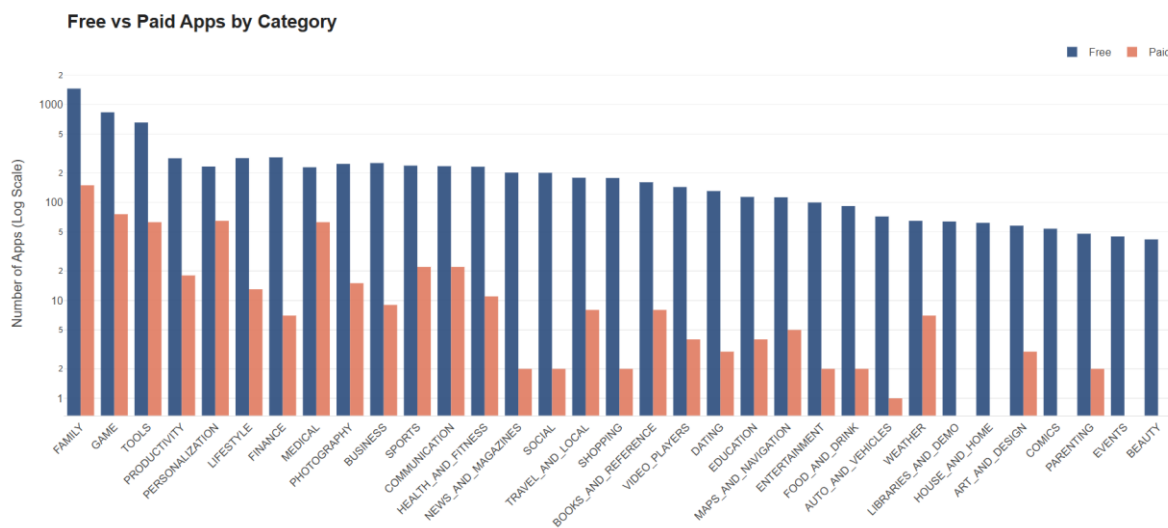
    margin=dict(t=100, b=50, l=60, r=40)
)

# we are going to remove the borders at the graph
g_bar.update_traces(marker_line_width=0, opacity=0.9)

g_bar.show()

```

Graph:



Interactive graph: [Free vs Paid Apps by Category](#)

The results clearly show that free apps dominate all categories in the Google Play Store, while paid apps account for only a small portion of the total.

However, this pattern is not uniform: certain categories have a relatively higher proportion of paid apps compared to others.

In particular, categories such as Medical, Personalization, and Tools stand out with a significantly higher number of paid apps, indicating greater monetization potential in these segments.

In contrast, categories such as Social, Shopping, and News are predominantly dominated by free apps, suggesting that users expect free access.

Additionally, categories with a high number of apps such as Family and Games also have the largest absolute number of paid apps, suggesting that larger and more competitive markets may offer greater monetization opportunities despite the overall dominance of free apps.

Overall, this distribution suggests that the viability of a strategy for paid apps depends to a large extent on the target category and the prevailing market norms within it.

Comprehensive and Descriptive Analysis of Financial Situation

We have seen the Distribution of paid and free applications in each Category.

But for new developers and investors who are willing to release new apps in play store, this leads to many more questions:

- How much should they charge?
- How much do other apps charge in that category?
- How much revenue could they make?
- And how many downloads might they lose because the app is paid?

Let's try and answer these questions with some Box plots.

Box plots provide a clear and effective way to visualize the distribution of a dataset. They summarize key descriptive statistics, including the median, minimum and maximum values, and the lower and upper quartiles.

By presenting this information graphically, box plots allow for quick comparisons between datasets and help identify patterns such as variability, central tendency, and potential outliers.

Creating Box Plots

We are going to design a box plot that shows the number of Installs for free vs paid apps. Two plots will be generated on the same Graph, one for free and one for paid apps.

No additional DataFrames need to be built, as the existing DataFrames can be efficiently passed as arguments to Plotly's box plot.

We are going to answer some questions like how the median number of installations compares and we are going to examine the effects of this difference.

Code for the box plot:

```
box = px.box(
    df_apps_clean,
    y='Installs',
    x='Type',
    color='Type',
    notched=True,
    points='all', # It should display all the install numbers
    title='<b>How Many Downloads are Paid Apps Giving Up?</b>', # Here, we are
    using html tag.
    template='plotly_white', # white background
    color_discrete_sequence=['#2C3E50', '#18BC9C'] # Dark blue and green palette
)

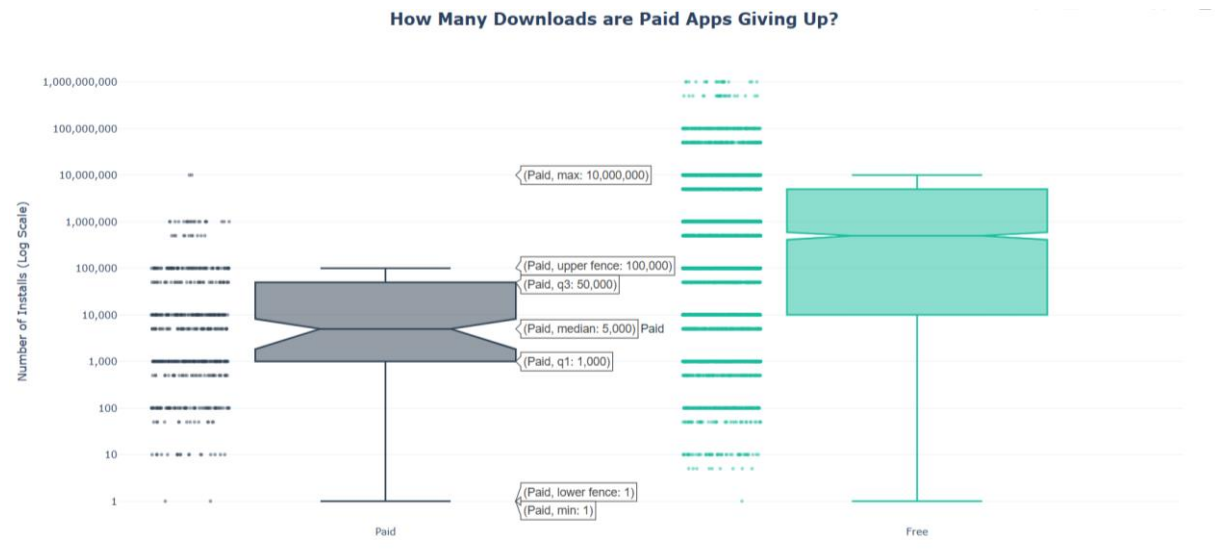
box.update_layout(
    yaxis=dict(
        type='log',
        title='Number of Installs (Log Scale)',
        showgrid=True,
        gridcolor='#EBF0F8', # Subtle gridlines
        zeroline=False,
        tickformat=', ' # Adds commas to y-axis numbers for readability
    ),
    xaxis=dict( title='', showgrid=False),
    title_x=0.5, title_font_size=20,
    showlegend=False,
    margin=dict(t=80, b=40, l=60, r=40),
    hoverlabel=dict(
        bgcolor="white", font_size=14, font_family="Arial"
    )
)

box.update_traces(
    marker=dict(opacity=0.5, size=4),
    line=dict(width=2),
    # Simplifying hover text:
    hovertemplate="<b>{%y:,.0f}</b> Installs<extra></extra>"
)

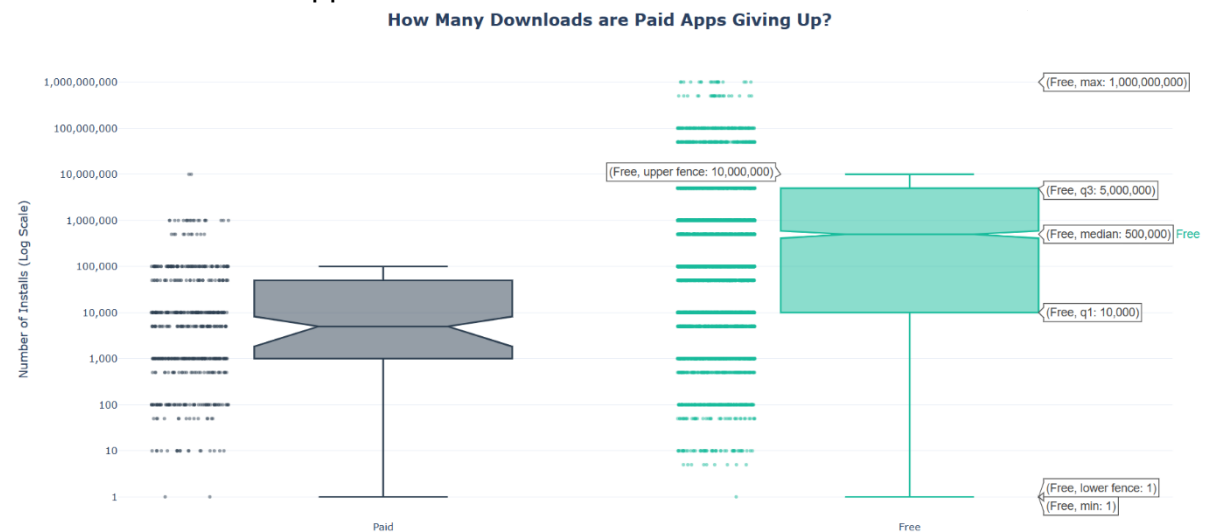
box.show()
```

Graphs:

With values for paid apps:



With values for free apps:



Interactive graph: [How Many Downloads are Paid Apps Giving Up?](#)

The box plots reveal a striking divergence in the distribution of installations between free and paid apps and highlight the significant impact of pricing on user preferences.

Free apps have a significantly higher average number of installations (around 500,000) than paid apps (about 5,000), suggesting that the removal of the price barrier drastically increases accessibility and acceptance.

Furthermore, the upper quartile and maximum values for free apps reach the range of tens or even hundreds of millions, suggesting that viral scalability with high growth is largely concentrated in the free segment.

In contrast, paid apps have a much narrower distribution with less variability, suggesting a more limited reach and slower growth potential. While paid apps can still generate direct revenue per user, the data clearly points to a trade-off: Developers who choose the paid model will see a significant decrease in the number of downloads.

Consequently, a free or a free trial strategy may be more effective in maximizing the user base and unlocking long-term monetization opportunities through alternative revenue streams such as advertising or in-app purchases.

Analysis of Paid Applications: Revenue by Category

To assess market viability and return on investment, it is essential to understand how revenue from paid apps is distributed across different categories.

In this section we are going to explore the estimated revenues from paid apps, with a particular focus on median revenue, as this is a more reliable indicator than average figures, which can be heavily influenced by extreme outliers.

By analyzing performance at the category level, we aim to assess whether typical app revenue is sufficient to offset standard development costs, which are often estimated at around \$30,000

Furthermore, we are going to point out the fluctuations within each category and identify segments in which a small number of high-performing applications generate asymmetrically high revenue.

Creating box plot for different branches:

We are going to create a box plot which shows the descriptive analysis for the estimated revenue of different categories of paid applications in Play Store.

Filtering the DataFrame for paid apps:

```
df_paid_apps = df_apps_clean[df_apps_clean['Type'] == 'Paid']
```

This line filters the dataset to include only entries where the app type is marked as “Paid,” creating a new DataFrame that contains exclusively paid applications to design the box plot.

Code for the box plot:

```
box = px.box(
    df_paid_apps,
    x='Category',
    y='Revenue_Estimate',
    title='Estimated Revenue of Paid Apps by Category',
    color_discrete_sequence=['#F75270']
)

box.update_layout(
    template='plotly_white', # white background
    title={
        'y': 0.95, 'x': 0.5,
        'xanchor': 'center', 'yanchor': 'top',
        'font': dict(size=20, color='#C80000', family="Helvetica, Arial Black,
sans-serif")
    },
    xaxis_title='App Category',
    yaxis_title='Revenue Estimate (USD, Log Scale)',
    xaxis_title_font=dict(size=13, family='Arial Black', color='#C80000'),
    yaxis_title_font=dict(size=13, family='Arial Black', color='#C80000'),

    xaxis=dict(
        categoryorder='max descending',
        tickangle=-45,
        showline=True,
        linewidth=1,
        linecolor='lightgrey'
```

```

),

yaxis=dict(
    type='log',
    showgrid=True,
    gridcolor='#f0f0f0',
    showline=True,
    linewidth=1,
    linecolor='lightgrey'
),

hoverlabel=dict(
    bgcolor="white", font_color="#C80000",
    font_size=14, font_family="Arial"
),

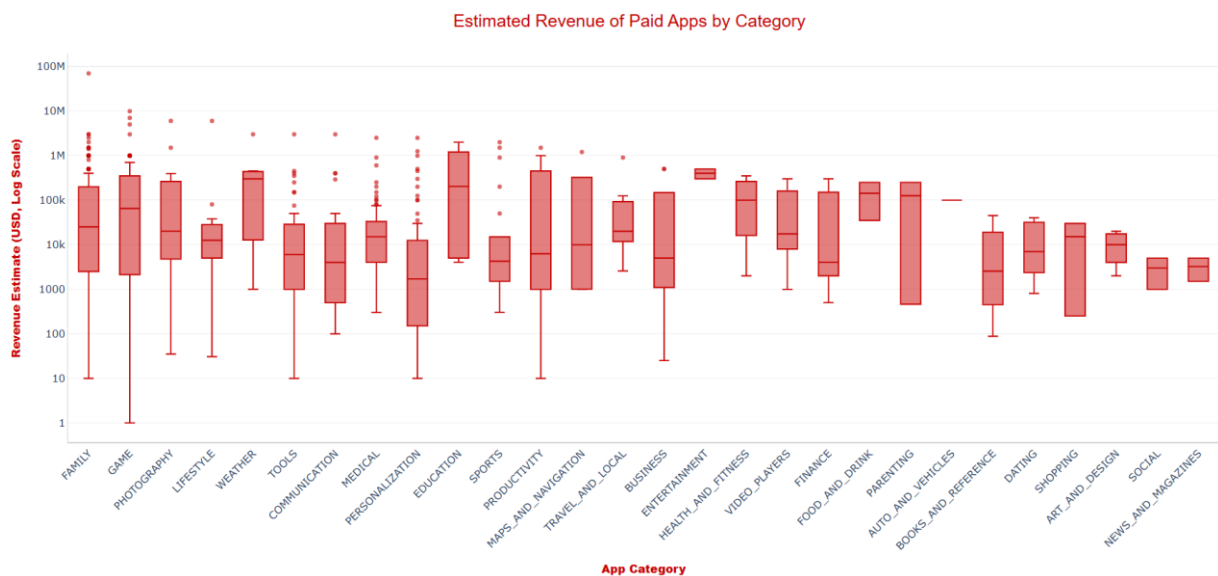
margin=dict(l=60, r=40, t=80, b=120)
)

box.update_traces(
    marker=dict(size=5, opacity=0.6, color='#C80000'),
    line=dict(width=1.5),
    hovertemplate="<b>{%y:,.0f}</b> Installs<extra></extra>"
)

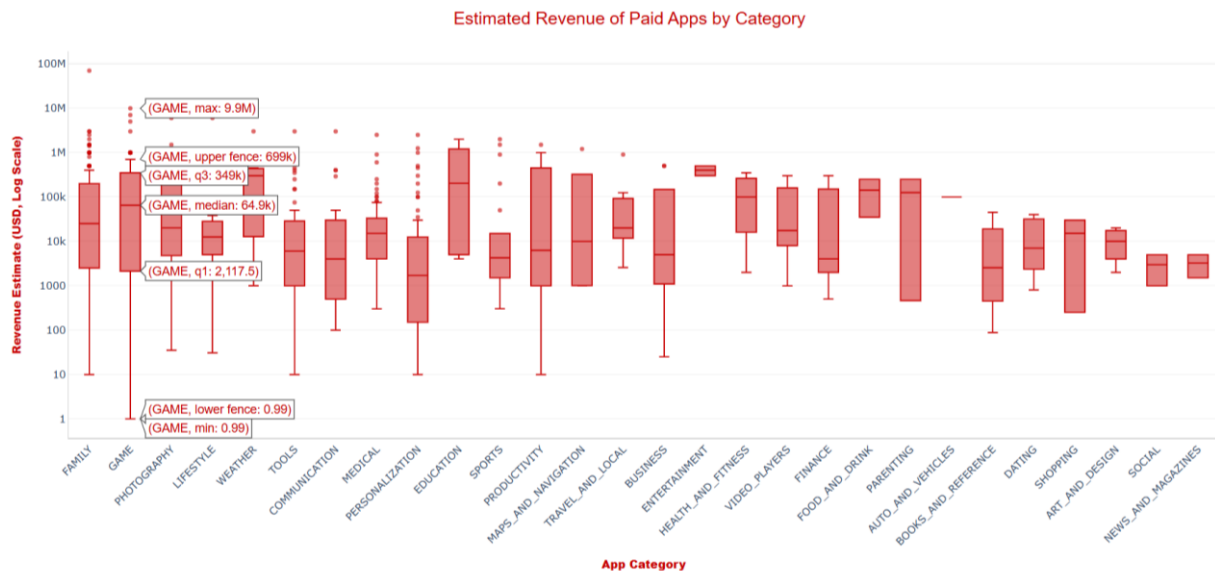
box.show()

```

Graph:



Example for “Game” category:



Full interactive graph: [Estimated Revenue of Paid Apps by Category](#)

The box plot shows significant oscillations in revenue across app categories, with highly irregular distributions and numerous outliers suggesting that a small number of apps generate disproportionately high revenue.

Categories such as Games, Education and Tools have higher average revenues and wider interquartile ranges, which indicates both a greater revenue potential and a greater unpredictability.

In contrast, categories such as Social, News & Magazine and Art & Design have relatively lower median values and narrower distributions, which indicates more restricted monetization opportunities for paid apps.

Overall, the logarithmic scale shows that while most paid apps generate only modest revenue, some powerful outliers significantly skew the top end of revenue in several categories.

Analysis of the Prices of Applications

In this section we are going to evaluate the pricing structure of paid apps within the dataset. We are going to start the analysis by determining the median price, which serves as a robust measure of central tendency and is less susceptible to outliers.

Building on this, by using box plots we are going to explore the price distributions across different app categories, enabling a comparative assessment of pricing strategies and variability within each category.

This approach provides insights into how app pricing varies by market segment and highlights potential patterns or outliers in the pricing of paid apps.

Descriptive Analysis for Paid Apps

```
df_paid_apps = df_apps_clean[df_apps_clean['Type'] == 'Paid']  
  
print(df_paid_apps.Price.describe())
```

count	589.00
mean	4.48
std	5.77
min	0.99
25%	1.99
50%	2.99
75%	4.99
max	79.99

The descriptive statistics show that most paid apps are relatively inexpensive: the median price is \$2.99, and 75% of the apps cost less than \$4.99.

However, the high standard deviation and the maximum price of \$79.99 suggest the presence of significant outliers, indicating that a small number of apps are significantly more expensive than the majority.

This suggests a right-skewed distribution, with most apps concentrated in the lower price segment, while a few premium offerings drive up the average price.

Overall, the pricing structure reflects a market dominated by affordable apps, with a few high-quality or specialized applications offered at significantly higher prices.

Creating Box Plot for the Distribution of Certain Parameters

To better understand pricing configurations across different segments, we are going to construct a new box plot to visualize the distribution of prices for paid applications by category.

Code for the box plot:

```
box = px.box(df_paid_apps,
             x='Category',
             y="Price",
             title='<b>Distribution of Prices of Paid Apps by Category</b>',
             template='plotly_white',
             color_discrete_sequence=['#748DAE'])

box.update_layout(
    font=dict(family="Arial, Helvetica, sans-serif", size=12, color="#333333"),
    title_x=0.5, title_font_size=18,
    xaxis_title='<b>App Category</b>', yaxis_title='<b>Price (USD)</b>',

    xaxis=dict(
        categoryorder='max descending',
        tickangle=-45, # Angles the category labels so they don't overlap
        showgrid=False),

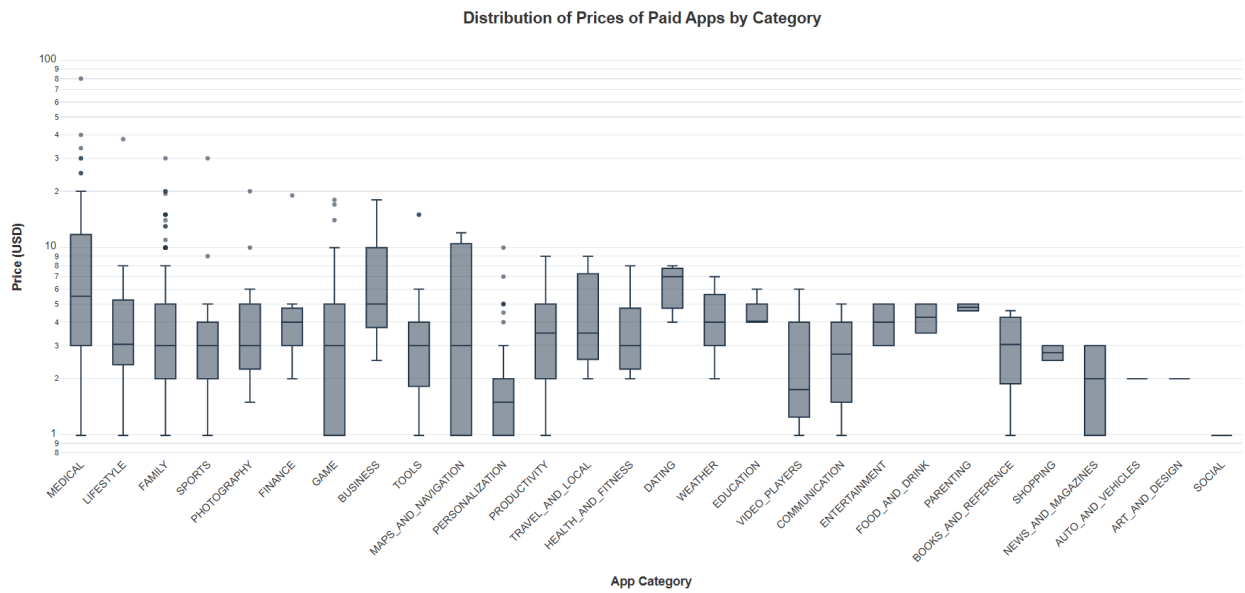
    yaxis=dict(
        type='log', showgrid=True, gridwidth=1,
        gridcolor='#E5E7EB', zeroline=False),

    margin=dict(t=80, b=80, l=60, r=40),
    showlegend=False,
    hoverlabel=dict(
        bgcolor="white", font_color="#213448",
        font_size=14, font_family="Arial"
    ))

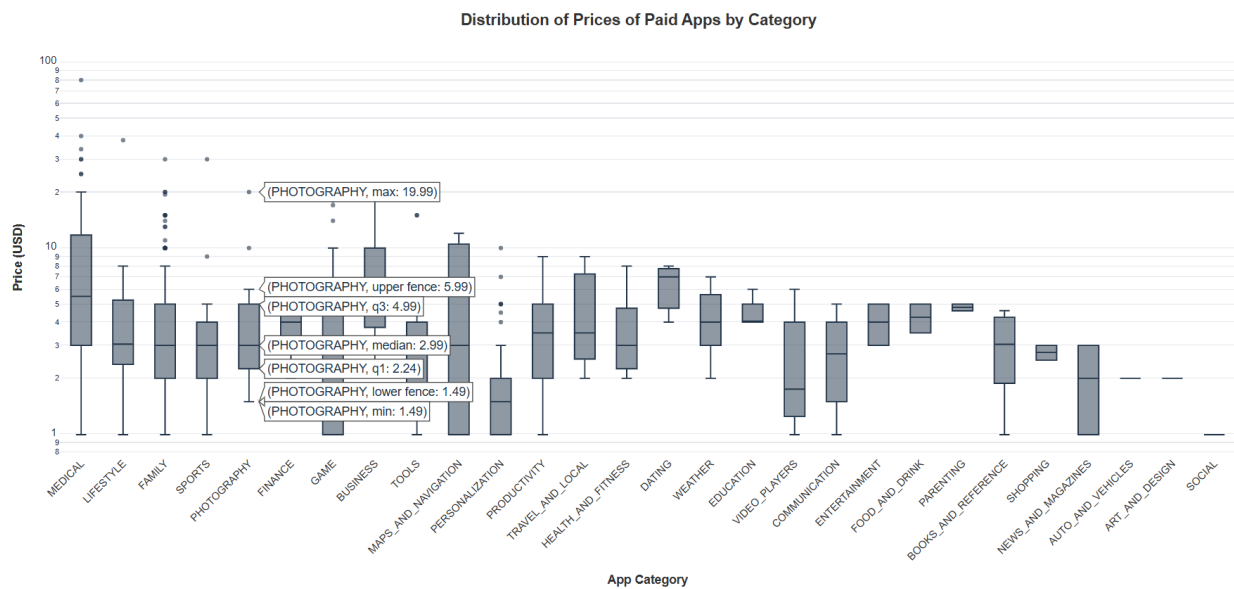
# Refine the look of the boxes and outlier points
box.update_traces(
    marker=dict(size=5, opacity=0.6, color='#213448'),
    line=dict(width=1.5),
    hovertemplate="<b>{y:,.0f}</b> Installs<extra></extra>")

box.show()
```

Graph:



Values for Category: Photography:



Interactive Graph Link: [Distribution of Prices of Paid Apps by Category](#)

The box plot shows significant price differences between app categories, suggesting that pricing strategies depend heavily on the specific segment.

Categories such as “Healthcare,” “Business,” and “Dating” exhibit higher median prices and wider interquartile ranges, suggesting both greater monetization potential and greater price dispersion within these segments.

In contrast, categories such as “Personalization,” “Communication,” and “Social” tend to have lower median prices and narrower distributions, suggesting a more price-conscious user base.

Overall, most categories continue to focus on a relatively affordable price range, but the variation varies considerably depending on the perceived value and use case of the respective category. This illustrates that users’ willingness to pay varies not only depending on the app’s functionality but also on its perceived necessity or specialization.

From a strategic perspective, these findings suggest that developers and investors should tailor their pricing models to users’ category-specific expectations and the respective value propositions.

While entering categories with a high median price offers greater revenue potential, it also requires the provision of specialized, high-quality solutions to justify the high prices. Conversely, categories with lower prices require competitive pricing and differentiation through features, user experience, or scalability.

A data-driven pricing strategy combined with clear value positioning will be crucial to successfully launching and establishing new applications in an increasingly competitive market.

Conclusion

This report provides a comprehensive analysis of apps in the Google Play Store, focusing on pricing models, the competitiveness of individual categories, download behavior, and revenue potential across various app genres. Throughout the project, in-depth expertise in data science was applied, starting with comprehensive data cleaning and preprocessing, including the removal of duplicates, the handling of nested structures, and the conversion of raw data into structured, analysis-ready DataFrames.

Particular attention was paid to creating clear and meaningful visualizations, with interactive charts created using Plotly—such as horizontal and vertical bar charts, pie charts and doughnut charts, scatter plots with color scales, and box plots—carefully designed to highlight patterns and insights. The descriptive statistical analysis also supported the interpretation of price distributions and category trends, thereby providing a deeper understanding of market dynamics. In addition, sophisticated data analysis methods, including sampling techniques and data transformations, were employed to ensure both the accuracy and clarity of the results.

Overall, this project demonstrates not only analytical thinking but also a strong ability to transform complex datasets into intuitive, visually appealing insights that can help developers and investors make strategic decisions.

References:

<https://www.udemy.com/course/100-days-of-code/>

https://pandas.pydata.org/docs/user_guide/index.html#user-guide

<https://matplotlib.org/stable/users/index.html>

<https://plotly.com/python/pie-charts/>

<https://plotly.com/python/getting-started/>

<https://plotly.com/python/box-plots/>

<https://plotly.com/python/horizontal-bar-charts/>

https://en.wikipedia.org/wiki/I_Am_Rich